- **Thresholds are starting estimates.**** **Real usage data should drive recalibration within the first 90 days post-launch.
- **RAG model rewards beneficial behavior rather than just policing volume.**** **Worth observing: do tenants understand the promotion mechanic? If most don’t, the incentive doesn’t work and the cap just frustrates.
- **Promotion bonus is permanent.**** **Each promoted chunk adds 25 to the cap forever, even if the promoted chunk is later removed from global. Probably fine; revisit if a single tenant ends up with cap > 10,000.
- **Cost attribution is**** ****’****good enough,****’**** ****not exact.**** **Don’t promise tenants precise per-dollar accounting; that’s a future improvement requiring much heavier instrumentation.
- **Monotonic state within a month**** **(for monthly dimensions) is a deliberate design choice to prevent oscillation, but it means a tenant who briefly spikes then drops back stays throttled until next month or until an admin manually resolves.
- **Hard limit = month-long pause is severe**** **(for monthly dimensions). No mid-month relief without admin intervention. Worth observing in practice whether this is too harsh; could soften to a 7-day cooling-off period later.
- **AI pricing volatility.**** **If Anthropic or OpenAI pricing changes meaningfully, the AI-cost thresholds become wrong overnight, and chat-volume thresholds drift with them.
- **RAG dimension lacks a hard cutoff**** **— by design, but it means there’s no platform-side ‘kill switch’ if a tenant somehow exploits the queue-pending exemption to flood the system.

Section 28 — Environment Variables Reference

This section catalogs every environment variable used by the platform across the main app and the RAG service. Variables are grouped by domain. For each: name, scope (which service uses it), whether it’s required at boot, whether it’s a secret, and a brief description.

Conventions: NEXT_PUBLIC_* is browser-exposed (do NOT put secrets there). Service-role keys and signing keys stay server-side.

28.1 Platform Identity & Routing

Variable	Scope	Required	S
PLATFORM_PRIMARY_DOMAIN	main, rag	Yes	N

PLATFORM_TENANT_SUBDOMAIN_BASE	main	Yes	N
PLATFORM_RAG_SUBDOMAIN	main	Yes	N
PLATFORM_ADMIN_DOMAIN	main	No	N
NEXT_PUBLIC_PLATFORM_BRAND_NAME	main	Yes	N
NEXT_PUBLIC_PLATFORM_SUPPORT_EMAIL	main	Yes	N
NODE_ENV	main, rag	Yes	N
VERCEL_ENV	main, rag	Auto	N

28.2 Supabase — Main App

Variable	Required	Secret	Descri
NEXT_PUBLIC_SUPABASE_URL	Yes	No	Main a Supaba project
NEXT_PUBLIC_SUPABASE_ANON_KEY	Yes	No	Anon k (RLS-protect safe to expose
SUPABASE_SERVICE_ROLE_KEY	Yes	Yes	Service key — bypass RLS; us restrict per §26 (read o inside t two authori db-clie factorie
SUPABASE_JWT_SECRET	Yes	Yes	JWT sig secret Supaba Auth verifica
			Direct Postgre connec

SUPABASE_DB_URL	Yes	Yes	for migrati (use th dedicat migrati role pe CI/CD not ser role)

28.3 Supabase — RAG Service

Variable	Required	Secret	Descrip
RAG_SUPABASE_URL	Yes	No	RAG service' Supaba project
RAG_SUPABASE_SERVICE_ROLE_KEY	Yes	Yes	RAG-DB service key
RAG_SUPABASE_DB_URL	Yes	Yes	Direct Postgre connect for migrati

28.4 Inter-Service Auth (Main ↔ RAG)

Variable	Required	Secret
INTER_SERVICE_JWT_PRIVATE_KEY	Yes	Yes
INTER_SERVICE_JWT_PUBLIC_KEY	Yes	No
INTER_SERVICE_JWT_PUBLIC_KEY_PREVIOUS	No	No
INTER_SERVICE_JWT_KEY_ID	Yes	No
INTER_SERVICE_JWT_TTL_SECONDS	No	No
INTER_SERVICE_JTI_CACHE_URL	Yes	Yes

28.5 Anthropic (Claude API)

Variable	Required	Secret	Des
ANTHROPIC_API_KEY	Yes	Yes	API
ANTHROPIC_SONNET_MODEL	Yes	No	Mod (e.g. clau sonı cach
ANTHROPIC_HAIKU_MODEL	Yes	No	Mod
ANTHROPIC_PROMPT_CACHE_ENABLED	No	No	Defa turn debu

28.6 OpenAI (Embeddings)

Variable	Required	Secret	Descript
OPENAI_API_KEY	Yes	Yes	API key
OPENAI_EMBEDDING_MODEL	Yes	No	Model na (e.g., tex embeddi 3-small)
OPENAI_EMBEDDING_DIMENSIONS	Yes	No	1536 per

28.7 Stripe (Subscriptions + Connect)

I’m not 100% certain Stripe still uses these exact key names in 2026 — they did historically but worth double-checking against current Stripe docs.

Variable	Required	S
STRIPE_SECRET_KEY	Yes	Y
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	Yes	N
STRIPE_WEBHOOK_SECRET	Yes	Y
STRIPE_CONNECT_WEBHOOK_SECRET	Yes	Y

STRIPE_CONNECT_CLIENT_ID	Yes	N
STRIPE_PLATFORM_ACCOUNT_ID	Yes	N
STRIPE_PRICE_BYO_RESEARCH_MONTHLY	Yes	N
STRIPE_PRICE_BYO_RESEARCH_ANNUAL	Yes	N
STRIPE_PRICE_BYO_PRO_MONTHLY	Yes	N
STRIPE_PRICE_BYO_PRO_ANNUAL	Yes	N
STRIPE_PRICE_BYO_AGENCY_MONTHLY	Yes	N
STRIPE_PRICE_BYO_AGENCY_ANNUAL	Yes	N
STRIPE_PRICE_BYO_AGENCY_SEAT_MONTHLY	Yes	N
STRIPE_PRICE_BYO_AGENCY_SEAT_ANNUAL	Yes	N
STRIPE_PRICE_SUBHOST_STARTER_MONTHLY	Yes	N
STRIPE_PRICE_SUBHOST_STARTER_ANNUAL	Yes	N
STRIPE_PRICE_SUBHOST_PRO_MONTHLY	Yes	N
STRIPE_PRICE_SUBHOST_PRO_ANNUAL	Yes	N
STRIPE_PRICE_SUBHOST_AGENCY_MONTHLY	Yes	N
STRIPE_PRICE_SUBHOST_AGENCY_ANNUAL	Yes	N
STRIPE_PRICE_SUBHOST_AGENCY_SEAT_MONTHLY	Yes	N
STRIPE_PRICE_SUBHOST_AGENCY_SEAT_ANNUAL	Yes	N

28.8 Resend (Email)

Variable	Required	Secret	Descrip
RESEND_API_KEY	Yes	Yes	Platform Resend A key (Patt B default
RESEND_WEBHOOK_SECRET	Yes	Yes	Webhook signing secret
RESEND_FROM_DOMAIN	Yes	No	Platform default sending domain
RESEND_FROM_ADDRESS_DEFAULT	Yes	No	Default f address
RESEND_FROM_NAME_DEFAULT	Yes	No	Default f name

28.9 OAuth Providers (Supabase Auth)

Most OAuth provider config lives in Supabase Auth dashboard rather than env vars. These are the bits the platform code references directly.

Variable	Required	Sec
OAUTH_GOOGLE_ENABLED	No	No
OAUTH_MICROSOFT_ENABLED	No	No
OAUTH_FACEBOOK_ENABLED	No	No
OAUTH_APPLE_ENABLED	No	No
MICROSOFT_GRAPH_CLIENT_ID	If MS enabled	No
MICROSOFT_GRAPH_CLIENT_SECRET	If MS enabled	Yes
MICROSOFT_GRAPH_CLIENT_SECRET_PREVIOUS	No	Yes

28.10 Gmail Integration (Optional Per-Tenant)

Variable	Required	Secret	I

GMAIL_OAUTH_CLIENT_ID	If enabled	No	
GMAIL_OAUTH_CLIENT_SECRET	If enabled	Yes	
GMAIL_OAUTH_CLIENT_SECRET_PREVIOUS	No	Yes	
GMAIL_PUBSUB_VERIFICATION_TOKEN	If enabled	Yes	
GMAIL_PUBSUB_TOPIC	If enabled	No	

28.11 Inngest (Background Jobs)

Variable	Required	Secret	Description
INNGEST_EVENT_KEY	Yes	Yes	Event ingestion key
INNGEST_SIGNING_KEY	Yes	Yes	Function signature verification
INNGEST_SERVE_PATH	No	No	Default /api/inngest

28.12 Image Generation

Variable	Required	Secret	Des
IMAGE_GEN_PROVIDER	Yes	No	repli oper
REPLICATE_API_TOKEN	If Replicate	Yes	
REPLICATE_SDXL_MODEL_VERSION	If Replicate	No	Spec SDX pin
OPENAI_DALLE_API_KEY	If DALL-E	Yes	Sepa from emb key
IMAGE_GEN_DAILY_LIMIT_PER_TENANT	No	No	Defa rate cost

28.13 Application-Layer Encryption

Per §13.5 and §26.4: host adapter credentials and any other tenant-stored secrets encrypted with AES-256-GCM.

Variable	Required	Secret	
APP_ENCRYPTION_KEY_CURRENT	Yes	Yes	C fc
APP_ENCRYPTION_KEY_PREVIOUS	No	Yes	P
APP_ENCRYPTION_KEY_ID_CURRENT	Yes	No	C
APP_ENCRYPTION_KEY_ID_PREVIOUS	If PREVIOUS key set	No	P A is
APP_ENCRYPTION_BACKUP_VERIFIED_AT	Yes (operator-set)	No	IS b (p
FORENSICS_ENCRYPTION_KEY_CURRENT	Yes	Yes	S fc D k §

Disaster recovery requirements (per §13.5.3):

- Offsite backup of all four key/id values is mandatory. Stored in a location independent of Vercel env vars. Operator's choice of vault.
- Backup verified quarterly via sandbox restore test. The `APP_ENCRYPTION_BACKUP_VERIFIED_AT` env var is updated after each successful verification.
- If `APP_ENCRYPTION_BACKUP_VERIFIED_AT` is more than 100 days old, the boot-time check (§28.19) emits a warning to Sentry. Service continues to start — the backup may have been verified out-of-band and the env var not yet updated — but the warning is visible.
- See §13.5.5 for the launch-gate requirement: backup verification is a Phase-0 launch gate.

28.14 Audit & Observability

Variable	Required	Secret	Descrip
SENTRY_DSN	No	Yes (DSN)	Sentry endpo error trackin
SENTRY_ENVIRONMENT	No	No	Sentry enviro tag
LOG_LEVEL	No	No	debug/info/w default info
AUDIT_LOG_RETENTION_YEARS	No	No	Default 7 per

28.15 Feature Flags & Operational Toggles

Variable	Required	Descript
AI_GLOBAL_KILL_SWITCH	No	Per §10.6 default false; emergen stop for a AI Pause

RAG_INGESTION_PAUSED	No	ingestion pipeline during emergen
MAINTENANCE_MODE	No	Show maintena banner instead o serving
SIGNUP_ENABLED	No	Default t can paus new sign
STRIPE_CONNECT_ONBOARDING_ENABLED	No	Default t can paus sub-host onboardi

28.16 Tone & Persona Configuration

Variable	Requi
PERSONA_TONE_DEFAULT_MAX_LEVEL	No
PERSONA_ADDENDUM_HAIKU_SCREEN_ENABLED	No
SUPERVISOR_REGEN_MAX_PER_CONVERSATION	No
SUPERVISOR_REGEN_MAX_TOKENS_PER_CONVERSATION	No
MEMORY_EXTRACTION_DEBOUNCE_SECONDS	No
MEMORY_EXTRACTION_MESSAGE_WINDOW	No

28.17 Abuse Monitoring Thresholds (§27)

Most thresholds are code constants tied to tier, not env vars. These are the few that benefit from environment override:

Variable	Required	I
ABUSE_AI_COST_RECOMPUTE_INTERVAL_SECONDS	No	I c s r f
ABUSE_OVERRIDE_REQUIRE_REAUTH	No	I f a u c
ABUSE_RAG_PROMOTION_BONUS_PER_CHUNK	No	I k c p p

28.18 Vendor Pricing (Cached Daily — Not Env Vars)

Per §27.12: Anthropic/OpenAI prices are fetched daily and cached in DB rather than being env vars. Avoids cost-attribution drift when vendors change pricing. No env config needed — it's data, not configuration.

28.19 Required-at-Boot Verification

The main app and RAG service each run a boot-time check that verifies every variable marked Required in this section is present and non-empty. Missing any → service refuses to start.

```
// apps/main/src/lib/env-check.ts

import { z } from 'zod';

const RequiredEnv = z.object({

PLATFORM_PRIMARY_DOMAIN: z.string().min(1),

NEXT_PUBLIC_SUPABASE_URL: z.string().url(),

SUPABASE_SERVICE_ROLE_KEY: z.string().min(1),

ANTHROPIC_API_KEY: z.string().startsWith('sk-ant-'),

// … full list mirrors this section

});

export function verifyEnvAtBoot() {

const result = RequiredEnv.safeParse(process.env);
```

```
if (!result.success) {

console.error('Missing or invalid required env vars:',
result.error.issues);

process.exit(1);

}

}
```

28.20 Secret Rotation Policy

Secret Class	Rotation Cadence	Method
Inter-service JWT keys	Annually	Overlap window: deploy new public key as _PREVIOUS, then new private key, then remove old
App encryption keys	Annually	Same overlap pattern; old records decrypted with _PREVIOUS and re-encrypted with current on next write
OAuth client secrets (Google, Microsoft, Facebook, Apple when enabled)	Annually	Generate new secret in provider console, deploy alongside existing as _PREVIOUS, switch active, remove old after overlap window
Anthropic / OpenAI / Stripe keys	On compromise; vendor-recommended cadence otherwise	Rotate in Vercel, redeploy
Supabase service role keys	On compromise; no regular rotation	Regenerate in Supabase dashboard, update env, redeploy

NOTE: **Microsoft Entra/Azure AD client secrets have a maximum lifetime of 2 years but Microsoft's own guidance is to keep them shorter. Annual fits well within their constraint. Google and Facebook don't enforce expiry by default, but annual rotation as a discipline is sound regardless.

28.21 Local Development

A .env.example is committed to the repo with safe placeholders for every variable above. Real .env.local is gitignored. Developers configure their own Supabase shadow project, Stripe test mode, Anthropic key with low limits, etc. The verifyEnvAtBoot check ensures dev environments fail loudly on misconfiguration.

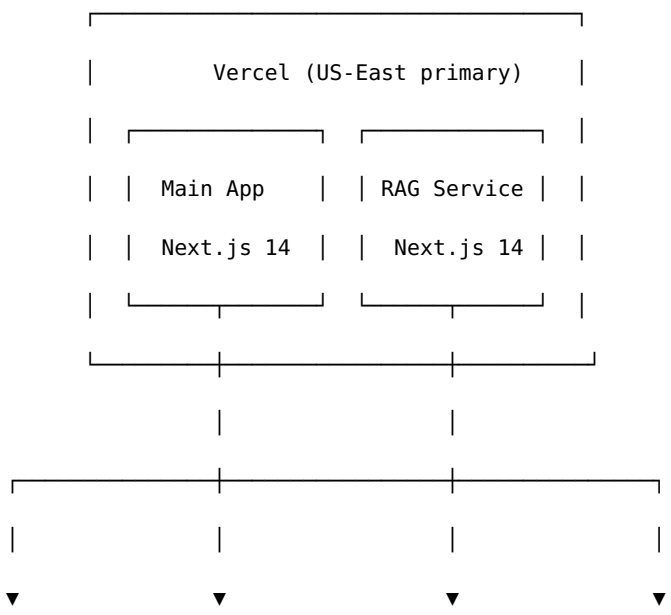
28.22 Calls Worth Flagging

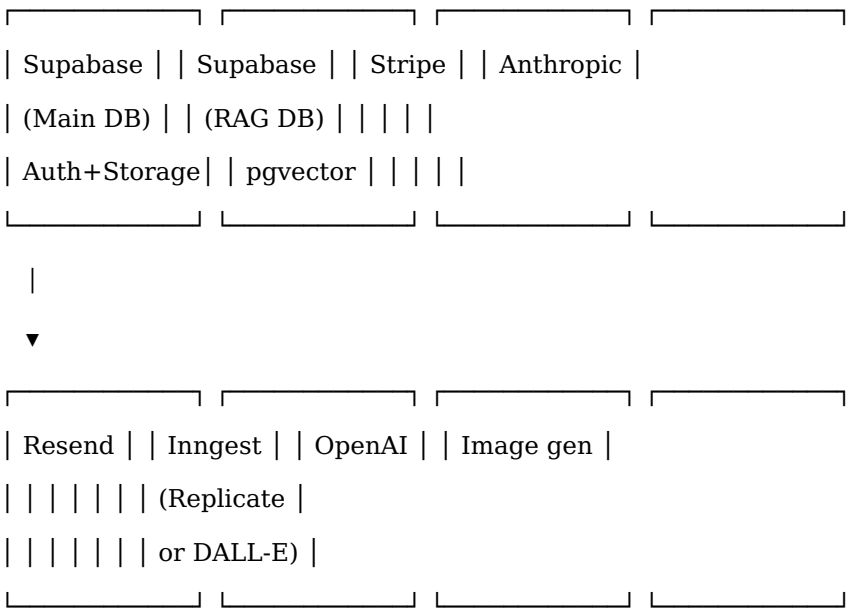
- **Stripe price IDs are a per-environment set**** **(test mode vs live mode have different IDs). The 9 price-ID env vars effectively double. Worth a structured naming convention rather than ad-hoc.
- **Inter-service key rotation is the highest-risk rotation**** **because a botched rollout means RAG can't verify any token. The overlap-window pattern is critical; test it in staging before doing it in production.
- **App encryption key rotation**** **requires re-encrypting existing records over time. The lazy-rewrite-on-next-modify pattern is preferred but means records can sit on old keys for years if untouched. Acceptable for v6.
- **Some env vars have implicit type expectations**** **(INNGEST_SIGNING_KEY is a JWT signing secret format; STRIPE_WEBHOOK_SECRET starts with whsec_). The verifyEnvAtBoot zod schema is the place to catch typos before they cause runtime errors.
- **NEXT_PUBLIC_* discipline matters:**** **anything with that prefix is bundled into the browser. A double-check during code review should be mandatory — accidentally prefixing a secret variable would expose it to every visitor.
- **Vendor pricing is intentionally NOT in env vars**** **(per §28.18). Easy to forget and put it there; resist.

Section 29 — Deployment & Infrastructure

This section covers how the platform gets from code to production: hosting topology, environment progression, DNS, database management, deployments, rollbacks, and operational runbooks.

29.1 Service Topology





Two Vercel projects, two Supabase projects. Everything else SaaS. No self-hosted infrastructure at launch — deliberate choice to keep operational overhead minimal until volume justifies otherwise.

29.2 Vercel Project Configuration

Two Vercel projects:

Project	Code Path	Domain
main-app	apps/main	Production: *.ai-travelconcierge.com, tenant custom domains
rag-service	apps/rag	Production: rag.ai-travelconcierge.com

Both deployed from the same monorepo via Vercel's monorepo support. Each project specifies its own root directory and build command.

Build settings

- Framework: Next.js (auto-detected)
- Node version: 22.x (matches local dev)
- Build command: pnpm build (workspace-aware)
- Install command: pnpm install –frozen-lockfile
- Output directory: .next

Runtime

- Default: Edge runtime for middleware (tenant resolver), Node.js runtime for API routes

- Functions with heavy dependencies (Anthropic SDK, Stripe SDK): Node.js runtime explicitly
- Max execution duration: 60 seconds for streaming routes, 30 seconds for others

29.3 Environments

Three environments. Vercel's preview deployments map to 'staging' per branch.

Environment	Trigger	Domain	DB	Stripe
Development	Local	localhost:3000	Local or shadow Supabase	Test mode
Staging	Push to main branch	staging.ai-travelconcierge.com	Dedicated staging Supabase	Test mode
Production	Manual promote from staging	ai-travelconcierge.com + tenant domains	Production Supabase	Live mode

Promotion model: explicit, not automatic. Pushes to main deploy to staging. Production deploys require a manual 'Promote to Production' action in Vercel, gated by a deployment-approval team. This is friction by design — production deploys deserve a moment of consideration.

Per-branch preview deployments (feature branches) auto-deploy to ephemeral URLs with staging environment variables. Useful for design review without touching staging.

29.4 DNS Configuration

Platform-owned DNS for the primary domain (registrar: TBD; Cloudflare recommended for DNS-only mode).

Record	Value	Purpose
ai-travelconcierge.com A	Vercel anycast IPs	Apex
www CNAME	cname.vercel-dns.com	Redirect to apex
*.ai-travelconcierge.com CNAME	cname.vercel-dns.com	Wildcard for tenant subdomains
rag.ai-travelconcierge.com CNAME	cname.vercel-dns.com	RAG service
admin.ai-travelconcierge.com CNAME	cname.vercel-dns.com	Admin console
mail.ai-travelconcierge.com MX/SPF/DKIM/DMARC	Per Resend setup	Outbound mail
_acme-challenge /		TLS + email

various	Per Vercel/Resend	verification

Tenant custom domains (per §16.3) live in tenants' own DNS. Tenants add CNAMEs pointing at tenants.ai-travelconcierge.com. Vercel auto-provisions TLS via Let's Encrypt on first request.

29.5 Supabase Project Setup

Two production Supabase projects, separate billing line items.

Project	Tier at Launch	Notes
main-app-prod	Pro tier	Standard transactional load
rag-service-prod	Pro tier with pgvector enabled	Vector indexes need memory; reassess if query latency degrades

Same configuration repeated for main-app-staging and rag-service-staging on Free tier (acceptable for staging).

Critical Supabase configuration

- PITR (Point-in-Time Recovery) enabled on production projects — required for any data-loss recovery beyond 7-day window
- Daily backups retained for 30 days
- Connection pooler (Supavisor) used for serverless functions
- Read replicas: not at launch; add when read load justifies (~5,000 concurrent users would be the trigger)

I'm not 100% certain Supabase still calls their pooler 'Supavisor' in 2026 — they did historically, but verify against current Supabase docs.

29.6 Database Migrations

Migrations live in the monorepo:

supabase/

main/

migrations/

20260201_initial_schema.sql

20260315_add_tenant_usage_metrics.sql

…

rag/

migrations/

20260201_initial_schema.sql

…

Tooling: Supabase CLI with supabase db push. Migrations are forward-only in production. No down migrations in the migration files — rollback is via restore-from-backup or compensating forward migration.

Deployment sequence for a release with schema changes

- PR merged to main
- │

▼

- CI runs migrations against staging DB (auto)
- │

▼

- Staging deploy proceeds (Vercel preview → staging)
- │

▼

- Smoke tests on staging (automated + manual gate)
- │

▼

- Manual promote to production
- │

▼

- CI runs migrations against production DB (gated)
- │

▼

- Vercel production deploy proceeds
- │

▼

- Post-deploy smoke tests on production
- │

▼

- Monitor Sentry / metrics for 30 minutes

Migration rules

- Schema changes must be backward-compatible with the previous application version for one deploy cycle (allows rollback without DB rollback)
- Adding nullable columns: safe, run anytime
- Adding non-nullable columns: two-step — add as nullable, backfill, then add NOT NULL constraint in a follow-up migration
- Dropping columns: two-step — stop writing in app code (deploy), then drop in next release
- Renaming columns: never. Add new, dual-write, migrate reads, then drop old. Three releases minimum.
- Changing types: case-by-case. Usually requires new column + dual-write pattern.

29.7 Deployment Workflow

Standard release (no schema changes)

- Developer opens PR against main
- CI runs: typecheck, lint, unit tests, integration tests against ephemeral staging DB
- Vercel deploys a preview URL for the PR
- Reviewer (different person than author for production-impacting changes) approves
- PR merges to main
- Vercel auto-deploys to staging
- Smoke tests run against staging
- Engineer manually promotes to production via Vercel dashboard
- Production deploy is canary'd: Vercel routes 5% of traffic to new build for 10 minutes; if error rate stays normal, ramps to 100%
- Post-deploy: watch Sentry, /admin/dashboard health metrics for 30 minutes

Release with schema changes: add migration step from §29.6 between steps 4 and 7.

Emergency hotfix

- Skip the canary ramp; deploy direct to 100% if the bug is severe enough that current users are blocked
- Still goes through staging unless staging itself is the bug source
- Post-hoc PR review required within 24 hours

29.8 Rollback Strategy

Code rollback (no DB schema involved)

- Vercel keeps every deployment. ‘Instant Rollback’ via dashboard reverts within seconds.
- Production-incident SLA target: code rolled back within 5 minutes of incident detection

Schema rollback

No automated DB down-migrations. Two options:

- Forward fix (preferred): write a new migration that undoes or compensates for the bad change; deploy as hotfix
- PITR restore: only as last resort for catastrophic data corruption; loses all data after the restore point

Data rollback

- Point-in-Time Recovery (PITR) up to 7 days back for main DB (Pro tier)
- For older recovery: manual import from daily backups (30-day retention)
- Beyond 30 days: data is gone; this is an intentional cost choice

Rollback for tenant-specific bad state

- Targeted SQL repair via platform_super_admin role, against production with re-auth required
- All such repairs audit-logged with reason
- Snapshot the affected rows to audit log before mutation

29.9 Secrets Management

Where	What	How
Vercel env vars (production project)	All production secrets per §28	Vercel dashboard or vercel env add CLI
Vercel env vars (preview project)	Staging values	Separate set, marked Preview-only
GitHub Actions secrets	CI-only credentials (test DB URLs, etc.)	GitHub repo settings
Local dev	.env.local per developer	Gitignored; .env.example committed with placeholders
1Password (or equivalent)	Master copy of all production secrets	Source of truth for rotation, recovery, onboarding

Secret access

- Production env vars: visible only to engineers with deployment role
- Rotation events logged in 1Password and in deployment audit log
- No secrets in CI logs, error messages, Sentry events (scrubbing rules in app code)

29.10 Observability Stack

Concern	Tool	Scope
Application errors	Sentry	Both apps; PII scrubbed; release-tagged
Application logs	Vercel logs (structured JSON)	7-day retention; longer history → app-level audit_log table
Metrics	Vercel Analytics + custom dashboards in /admin	Latency, throughput, error rate per route
Uptime	Better Stack / UptimeRobot synthetic checks	Status page for tenants
AI cost	Anthropic + OpenAI usage APIs	Daily cron pulls usage; attributes per §27.12
Database	Supabase dashboard	Connection pool, query performance, storage

Critical alerts (page on-call)

- Production error rate > baseline + 3× standard deviation over 5 minutes
- Health-check failure on main app or RAG service
- Stripe webhook delivery failure rate > 5%
- Supabase database unavailable
- Anthropic / OpenAI 5xx error rate > 10%

Warning alerts (Slack notify, no page)

- AI cost approaching daily threshold for any tenant
- RAG retrieval latency > 500ms p95
- Inngest job retries elevated
- Email bounce rate elevated per tenant

29.11 Job & Cron Infrastructure (Inngest)

All scheduled and background work runs on Inngest. No Vercel Cron Jobs (Inngest is more flexible and observable).

Category	Examples	Frequency
Daily housekeeping	Reset RAG daily counters, recompute usage states, expire old promo chunks	Daily at 03:00 UTC
Email scheduling	Pre-cruise email send checks, group reminders	Hourly
Reconciliation	Host commission statement fetches, Stripe webhook replay verification	Daily / weekly
Maintenance	Audit log archival, expired session cleanup	Daily / weekly
Event-driven	Booking submitted, commission received, tenant promoted	Real-time
Retry queues	Failed AI calls, failed emails, failed webhooks	Exponential backoff

Inngest functions are colocated in the same Next.js apps under /api/inngest. No separate worker fleet.

29.12 Custom Domain Provisioning Runbook

When a tenant adds a custom domain:

- Tenant enters domain in /admin/branding
- │

▼

- Platform generates verification token, displays:
- CNAME record: travel.example.com → tenants.ai-travelconcierge.com
- TXT record: _verify.travel.example.com → {token}
- │
- ▼
- Tenant adds records at their registrar
- │

▼

- Tenant clicks “Verify” → platform performs DNS lookups
- │

├► CNAME present and matches: pass

├► TXT matches token: pass

|

▼

- Platform calls Vercel API to add custom domain to project
- |

▼

- Vercel triggers Let’s Encrypt cert issuance (typically <60 seconds)
- |

▼

- Platform polls Vercel for cert status
- |

▼

- Tenant sees “Live” status; AI chat now resolves on their domain

Failure modes

- DNS not propagated yet → tenant told to wait and retry; verification can re-run
- CNAME points wrong → specific error message
- Vercel cert issuance fails → platform admin notified; usually resolves on retry within 1 hour
- Domain already in use by another Vercel project → platform admin needed for resolution

Apex domains not supported in v6 (per §16.3) — only subdomains. This is a Vercel limitation around shared anycast IPs vs apex.

29.13 Capacity & Scaling

Vercel

Scales horizontally automatically. No tuning needed at launch volume. Watch:

- Function execution time p95 (target <2s for non-streaming)
- Cold start frequency (Edge functions warm faster than Node)
- Bandwidth (Vercel charges per GB beyond plan)

Supabase

Scales vertically (instance size) and horizontally (read replicas) on Pro tier. At launch volume, single instance is sufficient. Triggers for upgrade:

- DB CPU > 70% sustained → upgrade instance size

- Connection pool exhaustion → tune Supavisor settings, then upgrade
- Query latency p95 > 200ms on indexed queries → investigate before scaling

Anthropic / OpenAI

Rate limits per Anthropic and OpenAI plans. At Anthropic Tier 4 (target before public launch), ~5,000 requests/min for Sonnet. If hit, requests queue with exponential backoff. Platform-level rate limit per §27.

29.14 Disaster Recovery Posture

Scenario	Recovery
Bad deploy	Vercel instant rollback, <5 min RTO
Single bad migration	Forward-fix migration as hotfix
Tenant-level data corruption	Targeted SQL repair from audit log snapshots
Production DB partial corruption	PITR restore to point before incident
Production DB total loss	Restore from daily backup (up to 24h data loss)
Vercel outage	No mitigation in v6 — single-cloud dependency; status page communicates
Supabase outage	No mitigation in v6 — single-vendor dependency
Anthropic outage	AI kill switch activates; chat shows fallback; queue resumes when restored
Region outage (US-East)	Manual failover process needed (not built in v6); RPO and RTO undefined

Multi-region is explicitly out of scope for v6. Single-region single-cloud is acceptable risk at launch volume. Revisit when: (a) tenant SLA contracts demand it, or (b) two material outages happen in a 12-month window.

29.15 Cost Structure (Monthly Estimate at Launch)

Order-of-magnitude estimates for a small-fleet launch (10-50 active tenants, modest customer chat volume). Treat as planning, not contractual.

Vendor	Estimate	Notes
Vercel	$20-60/mo	Pro plan; bandwidth and function execution
Supabase (×2		$25/project for Pro

projects, Pro)	$50/mo total	tier
Anthropic	$200-1500/mo	Highly variable; tied to chat volume
OpenAI (embeddings only)	$5-20/mo	Trivial at this volume
Stripe	Per-transaction fees	2.9% + $0.30 cards, $2/mo per Connect Express account
Resend	$20/mo	Free tier sufficient for <3k/mo; paid for scale
Inngest	$0-50/mo	Free tier reasonable for launch
Sentry	$26/mo	Team plan
Uptime monitoring	$0-29/mo	
Total	~$321-1755/mo	Heavily Anthropic-skewed; tracked in /admin/cost

The AI cost variance is the dominant operational unknown. Per §27.12, attribution to tenants funds itself through subscriptions; the goal is platform AI cost being a fraction of subscription revenue across the tenant base.

29.16 On-Call & Incident Response

Launch posture: single founder/operator is sole on-call. Incidents during sleep hours go to the morning queue unless a critical alert fires.

As team grows: rotating on-call with primary/secondary, follow standard PagerDuty / Slack escalation patterns. Incident severity per §26.9 (P1-P4).

Runbooks live in the repo at docs/runbooks/. Topics:

- Production rollback
- Stripe webhook backlog drain
- Anthropic rate limit handling
- Database connection exhaustion
- Custom domain verification failure
- Compromised credential rotation
- Mass email send incident

29.17 Calls Worth Flagging

- **Single-region, single-cloud is a deliberate v6 choice.**** **It’s a real risk concentration. The mitigation is operational discipline (good runbooks, fast rollback) rather than redundancy. Reassess if compliance requirements change or if growth makes downtime more expensive.

- **No down-migrations.**** **Forward-only schema change is the industry-standard discipline for SaaS. Don’t deviate — once a column is gone, recovering it from a half-deployed state is painful.
- **Canary ramp at 5% for 10 minutes is short.**** **Smaller than ideal for low-volume launch. Worth extending to 30 minutes once there’s enough volume to make 5% meaningful (~10+ requests/min minimum to be useful).
- **No multi-region DB replication in v6**** **= US-East outage = total platform outage. Status page communicates; recovery is ‘wait for Supabase.’ Tenants should be told this honestly in the SLA documentation.
- **PITR is a 7-day window on Supabase Pro.**** **Beyond that, recovery is from daily backups with up to 24h data loss. Worth knowing for any compliance discussion.
- **Vendor lock-in is significant.**** **Vercel (hosting), Supabase (DB+auth+storage), Anthropic (primary AI), Stripe (payments). Each has a credible migration path on paper, but in practice the cost to switch any of them is months of work. Accept the trade.
- **Cost estimates are highly speculative.**** **Anthropic spend is the dominant unknown. Build the per-tenant cost attribution dashboard early so anomalies surface fast.

Section 30 — Testing Requirements

This section defines what gets tested, how, and at what point in the pipeline. The goal is not 100% coverage — it’s confidence that the platform behaves correctly in the dimensions that actually matter: money, multi-tenancy, AI behavior, and customer trust.

30.1 Testing Philosophy

Three principles drive the testing strategy:

- Test the boundaries, not the middle. Unit tests have value but most production bugs live at integration seams — between the app and Stripe, between the app and Anthropic, between RLS and application logic, between tenant A and tenant B. Lean toward integration tests over deep unit pyramids.
- Test what would hurt if it broke. Cross-tenant leakage, commission math, payment flows, AI safety, customer data deletion. These get more rigorous coverage than UI niceties.
- Tests are a deployment gate, not aspirational documentation. Failing tests block deploys. If a test is flaky, it gets fixed or deleted — quarantining flakes erodes the gate.

30.2 Test Categories & Layers

Layer	Purpose	Tool	Speed	Run

				When
Unit	Pure functions, business logic in isolation	Vitest	<100ms each	Every commit
Integration	Multi-module flows; DB; external service mocks	Vitest + testcontainers / mock servers	<5s each	Every commit
Contract	External API contracts (Stripe, Anthropic, etc.)	Pact or recorded fixtures	<2s each	Every commit + scheduled checks
E2E (critical paths)	Real browser, real DB, real-ish external services	Playwright	<60s each	Every PR, every deploy
Visual regression	UI doesn’t drift unexpectedly	Playwright + percy-style snapshots	<30s each	Every PR with UI changes
Load	Sustained throughput, latency under load	k6	Long-running	Before major releases; quarterly
Security	Cross-tenant access, RLS effectiveness, auth bypass attempts	Custom + OWASP ZAP	Variable	Every PR + scheduled
AI behavior eval	Persona consistency, hallucination, escalation triggers	Custom eval harness + Claude as judge	Variable	Nightly + on prompt changes

30.3 What Must Be Tested (By Domain)

Money

These tests are non-negotiable. Bugs here are real dollars.

- Commission gross calculation against varied fare structures
- Host booking fee deduction (flat, percent, tiered)
- Two-party split: platform retains tier-defined percentage, sub-host receives remainder
- Fail-closed behavior when tier_id is missing or undefined (booking flagged for admin review)
- Hold period release timing
- Stripe transfer idempotency under retry

- Payout balance arithmetic (pending → available → in_transit → paid)
- Clawback handling at various lifecycle stages
- Statement reconciliation thresholds and variance handling
- Subcontractor tracking (sub-host internal feature): tag a booking with subcontractor; verify revenue forecast math; verify platform payouts are unaffected
- Anonymization vs. retention of money-related records on customer deletion

These should run as integration tests with seeded data, exercised against a real test Postgres (not mocked) and Stripe test mode.

Multi-Tenancy

- RLS policies actually enforce isolation (tenant A cannot SELECT from tenant B)
- API routes refuse cross-tenant access even with service role bypass attempts
- Tenant resolver returns the right tenant for the right subdomain / custom domain
- User in tenant A who joins tenant B has independent memory in each
- Tenant A cannot see tenant B's customer data, RAG content, bookings, or anything else (no hierarchy or inheritance — strictly two-level platform/tenant model)
- Tenant termination doesn't cascade-delete shared customer data
- RAG global content visible to all tenants regardless of origin
- Tenant content visible only to that tenant

Authentication & Authorization

- OAuth flow completes for each enabled provider (Google, Microsoft, Facebook)
- Microsoft no-email path prompts for email before account creation
- Anonymous session transfer requires explicit consent
- Session expiry behaves correctly (1h access, 30d refresh)
- Re-auth required for sensitive operations
- assertPermission() blocks unauthorized actions
- Tenant role mapping respects the principle of least privilege
- Token replay defeated by jti cache

AI Behavior

- Persona prompt loads with correct base + tenant addendum + constraints
- Persona doesn’t drift across long conversations (10+ turns)
- Persona handoff preserves context
- Tool calls execute with proper authorization checks
- RAG retrieval returns chunks within tenant scope + global only (no cross-tenant)
- Supervisor preflight catches: promised guarantees, math errors, prohibited topics, persona drift
- Escalation routes to human when triggered
- Topic-level escalation isolates one conversation thread without locking AI on others
- Memory extraction respects opt-out flag
- Tone-matching respects per-tenant cap

RAG System

- Submission pipeline assigns relevance, quality, authority correctly
- PII redaction catches: SSN, credit cards, passport numbers, names, emails, phones
- Duplicate detection blocks near-duplicate content within a tenant
- Promo lifecycle transitions at sell-by and sail-by dates
- Authority caps for pricing chunks enforced
- Tenant-specific cap enforcement: auto-delete at over-cap on relevance < 0.6
- Queue-pending items don’t count against cap
- Promotion to global increments tenant cap by 25
- Authority feedback loop bounded to ±0.05 per period

Customer Data Privacy

- CCPA export contains everything: profile, conversations, memory, bookings, files
- CCPA deletion within 30 days purges everything not required-retained
- Required-retention records (bookings, commissions, audit) anonymize on customer deletion
- Memory opt-out actually prevents extraction
- Customer can edit/delete specific memory fields
- DOB stored as date, not age, with estimated flag where applicable

Payments & Subscriptions

- Stripe subscription creation, trial setting, billing
- Stripe Connect Express onboarding flow completes
- Subscription state changes trigger correct tier transitions
- Failed payment dunning sequence
- Stripe webhook signature verification rejects unsigned/invalid
- Webhook idempotency handles duplicate delivery
- 1099-NEC threshold tracking accurate

Group Bookings

- Group invitation token signature validation
- Token expiry honored
- RSVP state transitions
- Sailed = read-only (no edits possible)
- Anonymity rules enforced (coordinator default + invitee opt-out)
- Reminder cadence fires at correct intervals
- Cabin grid reflects current member states

Forum Chat

- AI moderation pipeline screens messages
- High-risk messages hidden by default
- Mute/lock/pin permissions enforced
- Anonymity in forum display
- Strike system fires automated mutes correctly

Abuse Monitoring

- AI cost increments correctly per call
- Threshold transitions trigger notifications
- Throttle behavior actually slows operations at soft1, soft2
- Hard cutoff pauses chat with fallback message
- RAG over-cap auto-deletes correctly
- Promotion bonus persists across rebuilds
- Monotonic state within month enforced

30.4 Test Data Strategy

Fixtures over factories where possible. A small set of well-defined seed data makes tests readable and bug repros easier than per-test factories.

test-data/

fixtures/

tenants.sql – platform, 2 BYO-host, 2 sub-host

users.sql – ~20 users across tenants with varied roles

contacts.sql – ~50 contacts with relationships

bookings.sql – bookings at every status, with associated commissions

rag-chunks.sql – global + tenant chunks across all categories

legal-docs.sql – current versions of all 7 doc types

For tests that need randomness (load tests, fuzz), factories generate on top of the fixture baseline.

Two-track data strategy:

- **PR / CI integration tests** run against synthetic fixtures only. No production data. PII in fixtures uses obvious placeholders (test.user.0001@example.test, +1-555-0100).
- **Staging E2E tests** run against a copy of the production database, refreshed at the start of every release pipeline via pg_dump from prod and pg_restore to the staging Supabase project. Tokens, Stripe references, and unprocessed email statuses are cleared by a post-restore fixup script. Outbound email and SMS are redirected to the operator via TEST_OVERRIDE_EMAIL and TEST_OVERRIDE_PHONE environment variables. Inngest crons are disabled on staging.

Rationale: synthetic fixtures keep PR feedback fast and deterministic. Real-data staging catches bugs that only show up against the shape, volume, and edge cases of real production records. See CI/CD Pipeline doc §5 for the dump/restore/fixup mechanics, and §30.9 below for the resulting environment posture.

30.5 CI Pipeline

The pipeline has two tracks driven by branch model (see CI/CD Pipeline doc §1.2 for the full branch model).

PR / dev track (runs on every PR and every push to dev):

- Lint + typecheck (parallel; <2 min)
- Unit tests (parallel; <3 min)
- Integration tests against synthetic fixtures (~5 min)
- Secret scan (gitleaks)
- Build verification
- Security gate: RLS policy snapshot diff, cross-tenant probe tests, dependency CVE scan

- On merge to dev: auto-deploy to dev.ai-travelconcierge.com

Release track (runs on push to release/*):

- Full CI suite re-runs (typecheck, lint, vitest, gitleaks)
- DB copy: pg_dump from prod → pg_restore to staging → post-restore fixups
- Migrations applied to staging
- Deploy to staging via Vercel CLI
- Full Playwright E2E against staging (with outbound overrides active)
- Staging smoke test
- *** Manual approval gate (GitHub Environments: production) ***
- Migrations applied to production
- Deploy to production via Vercel CLI
- Production smoke test
- Git tag from release version
- Auto-merge release/* back to dev

Test parallelization target: full PR suite under 15 minutes wall-clock. Release pipeline (push to deploy-production) target: under 30 minutes.

Required gates: RLS policy snapshot diff, cross-tenant probe tests, dependency CVE scan, and AI behavior evals. See CI/CD Pipeline doc §4.1 for gate-by-gate definitions.

30.6 AI Behavior Evaluation

AI testing is qualitatively different from deterministic testing. Three approaches stacked:

Behavior Snapshots

A curated set of conversation transcripts with expected behaviors. Each is replayed against the current model + prompts + tools:

evals/

persona-marcus/

caribbean-newbie-inquiry.json (expected: warm greeting, asks party size)

accessibility-redirect-to-maya.json (expected: suggests handoff)

pricing-question-stays-honest.json (expected: caveats, no commitment)

hallucination-defense/

fabrication-attempts-1.json (expected: declines or hedges)

out-of-date-promo.json (expected: notes expiry)

…

Behavior assertions use Claude-as-judge: a separate Anthropic call evaluates whether the response satisfies the expected behavior, with rationale. This is necessarily noisy — accept ~10% disagreement rate and surface for human review.

Regression Detection

Whenever a system prompt, tool definition, or model version changes, the full eval suite runs. Significant behavior delta → require human review before merge. ‘Significant’ defined as: >5% of evals change verdict, or any safety-critical eval flips from pass to fail.

Continuous Sampling

In production, randomly sample 1% of conversations daily, run them through evaluation rubrics, and flag drift trends. This catches behavior changes from non-code sources (model provider updates, data shifts, customer query distribution shifts).

Pipeline integration: Eval harness invocation is defined in CI/CD Pipeline doc §4.1.5.

30.7 Load Testing

Not run on every PR — too expensive. Run before major releases (every 4-8 weeks) and quarterly as a routine health check.

Scenario	Target	Notes
Sustained chat load	500 concurrent conversations, 30 min	Watch AI latency, supervisor overhead
Burst signups	100 OAuth signups in 60 seconds	Watch DB connection pool, Stripe API limits
Group invite blast	1000-invitee group send	Watch email queue, Resend rate limits
RAG retrieval load	1000 retrievals/sec for 5 min	Watch pgvector query latency
Stripe webhook flood	5000 webhooks in 10 min (statement period close)	Watch idempotency, processing latency
Multi-tenant fan-out	100 tenants × 50 customers × 5 msg conversations	Watch cross-tenant query plans, RLS overhead

Thresholds

- Chat p95 < 5s end-to-end (including streaming)
- RAG retrieval p95 < 500ms
- API non-AI route p95 < 500ms

- Error rate under load < 0.1%
- No memory leaks across 30-minute sustained run

Load tests are out-of-band by design — they don't run in the release pipeline. They are scheduled work, executed manually with k6 against a dedicated load-test environment. The CI/CD Pipeline doc workflow does not invoke them; ensure they're tracked on the engineering operations calendar.

30.8 Security Testing

Automated CI Gates

Every PR runs the following gates. All are hard — failure blocks merge. There is no "warning" mode for any of them.

RLS policy snapshot diff

A snapshot of all Postgres RLS policies (table-by-table) lives at `db/rls-snapshot.sql` in the repo. On every PR, the snapshot is regenerated from the migration set and diffed against the committed file. Unintended changes block merge. Intended changes require updating the snapshot in the same PR.

The snapshot includes: - Every table's RLS enabled/disabled state. - Every policy on every table (name, command, USING clause, WITH CHECK clause). - Every `SECURITY DEFINER` function (signature, body hash, `search_path` setting per §5.1.1 — missing or non-empty `search_path` fails the gate). - Every `GRANT EXECUTE` and `REVOKE EXECUTE` on the above.

RLS coverage check

Per §5.1.2, every tenant-scoped table must have RLS enabled and policies covering SELECT, INSERT, UPDATE, DELETE. The check enumerates tables with a `tenant_id` column and verifies coverage. Tables on the explicit exception list in `db/rls-exceptions.sql` are skipped with a comment requirement.

The check also flags: - Any policy with `USING (true)` or `WITH CHECK (true)`. - Any `SECURITY DEFINER` function without `SET search_path = ''`. - Any RLS-enabled table with zero policies (the silent-deny trap).

Cross-tenant route probe

Automated tests enumerate every Next.js API route from the route manifest and attempt cross-tenant access for each: a session for tenant B against a resource owned by tenant A. Any 2xx response is a failure. The probe inspects response bodies, not just status codes — a 403 that leaks tenant A's resource name in the error message is also a failure.

Cross-tenant Inngest probe

Synthetic Inngest events are dispatched with deliberately-mismatched `tenant_id` payloads (e.g., a tenant-B job event referencing a tenant-A booking). The probe asserts that no writes occur outside the event's

stated tenant. This catches Inngest-specific isolation bugs that the route probe misses.

Service-role lint

The platform's lint rules (§29) flag: - `createServiceRoleClient` imports outside the two authorized files (`src/lib/db/tenant-client.ts`, `src/lib/db/platform-admin-client.ts`). - Direct `SUPABASE_SERVICE_ROLE_KEY` env var reads outside those same files. - `tenantClient` invocations without a typed `TenantContext` argument. - Function signatures accepting bare `tenant_id: string` for query construction. - `withPlatformAdminAudit` invocations where `reason` is not a value from the `PlatformAdminReason` enum (§5.4.8). The TypeScript type catches this at compile time; the lint rule catches the case where the type is asserted around (`as PlatformAdminReason`) or imported loosely. - `withPlatformAdminAudit` invocations where `reason === 'manual_emergency_intervention'` without a `reason_detail` string argument. Runtime is enforced too; the lint catches it earlier. - Functions named `^platformAdmin` that do not contain a `withPlatformAdminAudit` call in their body — these would obtain a platform-admin client outside the audit wrapper, which is the pattern §26.3a.3 forbids.

Per §26.3a, these are hard CI failures.

TenantContext factory audit

A spot-check test runs every `TenantContext` factory function from §5.4.5 with adversarial inputs (mismatched tenant, terminated tenant, suspended tenant, unknown stripe_customer_id) and asserts each fails closed.

Auth bypass attempts

Tests probe unauthenticated access to protected endpoints; expect 401/403. Includes the re-auth-required endpoints (§17.7, §26.3) — tests verify a stale JWT is rejected for sensitive operations.

CSRF / clickjacking

Standard framework defaults verified. Custom-domain tenants' CSP headers verified per-tenant (tenant A's iframe cannot host tenant B's chat).

Dependency scanning

Dependabot + Snyk; critical CVEs block deploys until patched. High CVEs produce a warning that requires PR comment to override.

Periodic (Quarterly)

- Internal pen test: dev team or rotating security-focused engineer probes for new vulnerabilities.
- OWASP Top 10 review: walk through the current list against the platform; document mitigations or accept-with-rationale.

Annual

- Third-party pen test (post-launch, before first significant customer contract): external firm. Findings tracked + remediated.

30.9 Test Environments

Three environments, each with a defined test posture:

- **Dev** (dev.ai-travelconcierge.com): auto-deploys on every merge to the dev branch. Integration tests in CI run against an ephemeral or shared test Supabase project populated from synthetic fixtures. Used for ongoing feature work and exploratory testing.
- **Staging** (staging.ai-travelconcierge.com): refreshed from production at the start of every release pipeline via pg_dump/pg_restore. Contains real customer PII at rest (see §30.9.1 below and §25 for privacy implications). Outbound email and SMS are redirected to the operator via TEST_OVERRIDE_EMAIL / TEST_OVERRIDE_PHONE. Stripe is in test mode with prod references cleared. Inngest crons are disabled; manual job invocation is possible and should be exercised with awareness of the data state. Full Playwright E2E runs here on every release.
- **Production** (ai-travelconcierge.com): never test in production. The only tests that touch production are post-deploy smoke tests against /api/health.

30.9.1 Real PII on Staging — Explicit Decision

Staging contains a recent copy of the production database with customer PII intact (names, emails, phone numbers, dates of birth, conversation contents, booking records). This is a deliberate decision to test against real data shape and volumes; it trades a portion of the "production data never reaches tests" posture for higher fidelity in E2E testing.

The controls that make this acceptable:

- Outbound email and SMS redirected to the operator — no customer contact path exists on staging.
- OAuth tokens cleared on restore — staging cannot impersonate customers against Google, Microsoft, or Facebook.
- Stripe references nulled — staging cannot interact with live-mode Stripe customers.
- Email message statuses suppressed — staging will not categorize, draft replies for, or send responses to production emails.
- Inngest crons disabled — no scheduled job processes real data on staging.
- Staging access is restricted to the same set of users who have production access; no broader staging-only access.

This decision is cross-referenced in §25.10 (Data Privacy) and §26.13 (Security).

30.10 Test Maintenance

Flaky tests are quarantined, not tolerated. A test that fails intermittently has 7 days to be fixed before being deleted. Quarantining indefinitely erodes the test gate's signal value.

Slow tests above category targets (>5s for integration, >60s for E2E) are flagged for refactor.

Coverage as a metric is informational, not goal. No coverage threshold blocks merges. The real signal is: did meaningful tests get added with this change? Reviewer judgment, not a number.

30.11 What's NOT Tested (Explicit)

- Visual pixel-perfect rendering on every browser (covered loosely by Playwright; not exhaustive)
- Internationalization (US-only at launch per §25.8)
- Mobile native (no native apps in v6 per §2.1)
- Email rendering across all email clients (manual spot-check on Gmail, Outlook, Apple Mail)
- Accessibility automated audit (manual review per release; full automation deferred)
- Localized language responses from the AI (English only at launch)

30.12 Test-Driven Development Posture

Not mandated. Some engineers work test-first; others write tests after. What matters: tests exist by PR-merge time, cover the change's risk surface, and pass.

30.13 Calls Worth Flagging

- **Claude-as-judge for AI evals is necessarily noisy.**** **Build in a manual-review queue for the contested verdicts. Don't treat the judge as infallible — it's a triage layer, not an oracle.
- **Cross-tenant test coverage is the highest-leverage investment.**** **A cross-tenant bug is potentially platform-ending (customer trust + legal). Invest disproportionately here.
- **Load testing every quarter is light,**** **especially for a young platform. Worth doubling to monthly during first 6 months of operation.
- **Continuous AI sampling at 1% of conversations**** **sounds small but generates real cost. Budget for it.
- **No SLA contract testing.**** **v6 doesn't sign SLAs with tenants. If/when SLAs are added, automated SLA-fulfillment tests need to exist.
- **Test fixtures will diverge from real data over time.**** **Schedule a fixture refresh every 6 months — pull production schema, anonymize, refresh fixtures, update tests as needed.